Báo cáo kỹ thuật: Xếp thời khóa biểu học phần bằng CSP

Tóm tắt

Bài tập lớn xây dựng ứng dụng xếp thời khóa biểu cho một tuần học điển hình từ danh sách buổi học, phòng, giảng viên và nhóm sinh viên. Bài toán được biểu diễn thành **Constraint Satisfaction Problem (CSP)**; dự án cài đặt hai bộ giải: quay lui theo thứ tự cố định và quay lui kết hợp **Minimum Remaining Values (MRV)**, degree heuristic và **forward checking**. Một bộ kiểm tra độc lập xác nhận mọi lịch được hiển thị hoặc xuất ra đều thỏa ràng buộc cứng. Ứng dụng có giao diện Streamlit, chức năng điều chỉnh giờ rảnh, giải thích một khung giờ đề xuất, chấm điểm lịch và tìm kiếm cục bộ để giảm điểm phạt. Trên dữ liệu mô phỏng 9–40 buổi học, thuật toán cải tiến giảm số nút xét; dữ liệu ràng buộc chặt cho thấy số lần quay lui giảm, nhưng chi phí cập nhật miền vẫn làm thời gian chạy cao hơn ở nhiều bộ khả thi. Kết quả này chỉ phản ánh các bộ dữ liệu đã thử, chưa đại diện cho lịch thật của trường.

1. Bài toán

Mỗi buổi học phải được gán một ngày, tiết bắt đầu và phòng. Dữ liệu gồm sĩ số, thời lượng 1–2 tiết, giảng viên, nhóm sinh viên, phòng, giờ rảnh và giờ ưu tiên. Các điều kiện bắt buộc là: phòng đủ chỗ và đúng loại; giảng viên rảnh trong toàn bộ buổi; buổi học không vượt cuối ngày; hai buổi giao thời gian không được dùng cùng phòng, giảng viên hoặc nhóm. Sau khi có lịch hợp lệ, hệ thống muốn giảm số tiết muộn, tiết trống giữa các buổi của một nhóm và mức phạt do xếp sai giờ ưu tiên.

Sản phẩm là công cụ demo cho người lập lịch ở quy mô nhỏ. Nó không quyết định số lớp cần mở, phân công giảng viên hay xử lý lịch tuần chẵn/lẻ.

2. Biểu diễn tri thức và mô hình CSP

Biến và miền

Với mỗi buổi học `S_i`, tạo một biến `X_i`. Giá trị của `X_i` là bộ `(d, p, r)` với ngày `d`, tiết bắt đầu `p` và phòng `r`. Một buổi dài `l_i` tiết sử dụng các tiết `p, ..., p+l_i-1`. Miền ban đầu `D_i` chỉ gồm các bộ mà phòng chứa đủ sinh viên/đúng loại, thời lượng nằm trong ngày và giảng viên rảnh trong tất cả tiết của buổi.

Ràng buộc nhị phân

Cho hai buổi `S_i, S_j`, hai khoảng giờ giao nhau nếu cùng ngày và `p_i < p_j+l_j` và `p_j < p_i+l_i`. Khi hai khoảng giao nhau, phép gán không hợp lệ nếu `r_i=r_j`, hoặc hai buổi có cùng giảng viên, hoặc cùng nhóm sinh viên. Bộ giải kiểm tra ràng buộc khi mở rộng một phép gán; bộ kiểm tra độc lập kiểm tra lại toàn bộ lịch trước khi chấm điểm/xuất CSV.

Hàm chất lượng

Với lịch hợp lệ `L`, điểm phạt:

`C(L) = w_late × late_periods + w_gap × student_gaps + w_pref × preference_penalty`.

`student_gaps` chỉ tính các tiết trống giữa buổi đầu và buổi cuối trong cùng ngày của từng nhóm. `preference_penalty` là tổng phạt của các tiết mà giảng viên được xếp, nếu file ưu tiên có khai báo. Điểm thấp hơn là tốt hơn; lịch có vi phạm cứng bị loại bất kể điểm này.

3. Thuật toán

A. Quay lui cơ bản

Chọn buổi chưa gán đầu tiên theo thứ tự dữ liệu, thử từng giá trị miền theo thứ tự ngày–tiết–phòng. Nếu giá trị xung đột với một buổi đã gán, bỏ qua. Khi không còn lựa chọn, quay lui về biến trước. Đây là mốc so sánh, không có suy diễn trên miền của các biến chưa gán.

B. Quay lui cải tiến

Trước mỗi bước, chọn biến có ít giá trị còn lại nhất (**MRV**). Khi hòa, ưu tiên biến liên quan với nhiều biến chưa gán hơn (**degree heuristic**). Sau một phép gán, **forward checking** loại các giá trị xung đột khỏi miền của những biến liên quan; nếu miền nào rỗng, quay lui ngay. Các giá trị bị xóa được lưu và khôi phục chính xác khi thử nhánh khác. Ca kiểm thử `test_enhanced_restores_domains_after_backtracking` xác nhận bước khôi phục này cần thiết để không báo sai một lịch có nghiệm là vô nghiệm.

```
search(assignment, domains):
    if all sessions assigned: return assignment
    x = first unassigned (A) or MRV + degree (B)
    for value in current domain[x]:
        if value conflicts with assignment: continue
        assign x = value
        if B: remove conflicting values from neighbor domains
        if every unassigned domain nonempty:
            result = search(assignment, domains)
            if result exists: return result
        restore removed values; unassign x
    return no solution in this branch
```

Trạng thái kết thúc: `FEASIBLE` là đã có lịch hợp lệ; `UNSAT` chỉ khi đã duyệt hết các nhánh; `TIMEOUT` khi hết thời gian trước khi chứng minh được kết quả. Việc dựng miền cũng nằm trong giới hạn thời gian của bộ giải.

C. Cải thiện lịch

Sau khi có nghiệm, tìm kiếm cục bộ thử (1) chuyển một buổi sang giá trị khác và (2) hoán đổi vị trí hai buổi. Chỉ giữ thay đổi có `C(L)` thấp hơn và được bộ kiểm tra độc lập xác nhận hợp lệ. Giao diện hiển thị điểm trước và sau. Phần này tối ưu chất lượng của một lịch đã tìm được; nó không thay đổi định nghĩa `UNSAT`.

4. Thiết kế ứng dụng và dữ liệu

Mã nguồn chính ở `timetable_ai/core.py`; `__main__.py` cung cấp CLI; `app.py` là giao diện Streamlit. Dữ liệu đầu vào được lưu ở `data/` với các file `sessions.csv`, `rooms.csv`, `availability.csv`, `preferences.csv` (tùy chọn) và `config.json`. Script `generate_data.py` tạo lại bộ mẫu bằng seed cố định; `run_experiments.py` tạo dữ liệu tạm, chạy hai bộ giải và ghi kết quả vào `results/benchmark.csv`.

Giao diện cho phép tải dữ liệu hoặc dùng bộ demo, thay đổi giờ rảnh của giảng viên trong phiên, chọn bộ giải và trọng số, xem thời khóa biểu theo nhóm/giảng viên/phòng. Người dùng có thể chọn một buổi và khung giờ đề xuất để thấy phòng thiếu chỗ, giảng viên không rảnh hoặc buổi nào gây xung đột. Lịch xuất dạng CSV chỉ khi qua bộ kiểm tra độc lập.

5. Thực nghiệm

Thiết lập

- Dữ liệu mô phỏng thông thường: `small` (9 buổi), `medium` (24 buổi), `large` (40 buổi). Dữ liệu ràng buộc chặt: `tight20` (20 buổi) và `tight40` (40 buổi), mỗi nhóm bốn buổi dùng một loại phòng riêng; ba buổi chỉ có một tiết khả dụng và buổi còn lại phải chiếm tiết còn trống. Mỗi bộ có 3 seed `3161`–`3163`.
- Mỗi seed chạy 5 lần cho từng thuật toán, cùng giới hạn 10 giây; thêm hai bộ riêng `backtracking` và `unsat`, mỗi bộ 5 lần/thuật toán. Tổng **170 phép đo**.
- Môi trường ghi trong CSV: Python 3.14.4, Linux/WSL2. Bảng dưới dùng trung bình 15 phép đo cho mỗi cặp bộ dữ liệu–thuật toán. Lần đo gốc được ghi lúc `2026-10-16T03:35:07+00:00`; số liệu thời gian có thể thay đổi khi chạy lại.
- Chỉ số: trạng thái, thời gian, số nút, số lần quay lui, số giá trị bị cắt, số vi phạm cứng và điểm phạt.

Kết quả trên năm bộ dữ liệu khả thi

Quy mô	Thuật toán	Thời gian TB (ms)	Số nút TB	Quay lui TB	Điểm phạt TB
9 buổi	Quay lui cơ bản	0,930	72,3	0	21,3
9 buổi	MRV + FC	1,080	9,0	0	24,0
24 buổi	Quay lui cơ bản	4,670	484,7	0	48,0
24 buổi	MRV + FC	8,434	24,0	0	48,0
40 buổi	Quay lui cơ bản	11,859	1.361,0	0	76,7
40 buổi	MRV + FC	25,401	40,0	0	67,3
tight20	Quay lui cơ bản	0,393	46,7	17,7	0
tight20	MRV + FC	0,543	20,0	0	0
tight40	Quay lui cơ bản	1,206	88,7	32,3	0
tight40	MRV + FC	1,840	40,0	0	0

Trong năm bộ này, cả hai thuật toán đều có lịch hợp lệ, **0 vi phạm cứng**. MRV + forward checking giảm số nút xét; ở `tight20` và `tight40`, nó loại bỏ hoàn toàn quay lui của bộ giải cơ bản. Dù vậy thời gian đo được vẫn cao hơn do chi phí quản lý miền. Điểm phạt khác nhau vì hai thuật toán trả nghiệm hợp lệ đầu tiên theo thứ tự tìm kiếm khác nhau; bảng không gồm bước tối ưu cục bộ. Các bộ `tight` cố tình tách phòng theo loại và hạn chế giờ rảnh, nên chỉ dùng để kiểm tra cơ chế thuật toán, chưa thể xem như lịch vận hành thực tế.

Hai trường hợp tập trung

- `backtracking`: quay lui cơ bản xét 6 nút và quay lui 1 lần; bản cải tiến xét 3 nút, không quay lui. Ca này minh họa tác dụng của MRV khi một buổi chỉ có một khung giờ.
- `unsat`: quay lui cơ bản xét 53.823 nút và quay lui 12.901 lần; bản cải tiến xét 3 nút, quay lui 1 lần. Cả hai báo `UNSAT`. Đây là một ca vô nghiệm được thiết kế có chủ đích, chưa đại diện cho mọi ca vô nghiệm.

6. Kiểm chứng và giới hạn

15 kiểm thử tự động đã đạt: hai bộ giải tạo lịch hợp lệ; phát hiện các loại xung đột và dữ liệu sai; tách `UNSAT`/`TIMEOUT`; khôi phục miền sau quay lui; xác nhận bộ `tight20` có nghiệm và giúp giảm quay lui; chặn xuất lịch khi giảng viên không rảnh ở tiết thứ hai; kiểm tra hoán đổi cải thiện lịch; xuất–nạp CSV. Luồng Streamlit mở demo → giải → đổi giờ rảnh → chạy lại và so sánh đã được chạy bằng AppTest. PDF báo cáo tiến độ được tạo từ mã kiểm thử và CSV thực nghiệm.

Giới hạn: dữ liệu vẫn là mô phỏng; các bộ thông thường chưa tạo nhiều tình huống phải quay lui, còn các bộ ràng buộc chặt có cấu trúc nhân tạo; tìm kiếm cục bộ không bảo đảm tìm cực tiểu toàn cục; dấu vết miền rỗng không phải tập nguyên nhân vô nghiệm tối thiểu. Việc áp dụng vào lịch thật cần thêm quy tắc thực tế và dữ liệu đã được giảng viên xác nhận.

7. Tái lập kết quả

Từ thư mục dự án:

```
python3 generate_data.py
python3 -m unittest discover -s tests -v
python3 run_experiments.py
python3 -m timetable_ai solve --data data/demo --solver enhanced --output schedule.csv
uv run --with streamlit==1.66.0 streamlit run app.py
```

Xem thêm README.md để biết cấu trúc dữ liệu và cài đặt thư viện.

Tài liệu tham khảo

1. Bài giảng môn học: L6 — Thỏa mãn ràng buộc và L4 — Tìm kiếm với tri thức bổ sung.
2. AIMA Python — csp.py, tham khảo thuật toán; repository sử dụng giấy phép MIT.
3. Google OR-Tools — CP-SAT, tham khảo mô hình ràng buộc, không dùng làm bộ giải trong dự án.

Mã thuật toán của dự án được viết riêng cho bài toán này; không sao chép module AIMA hoặc dùng OR-Tools để giải.